Rapport du Commandeur Data :
Sous-système PSMM (Python, Shell, Mariadb, Mail)

Objectif du projet :

Récupération des logs des serveurs FTP, SQL, Web et d'archiver les tentatives d'accès avec un compte ou un mot de passe non valide dans des bases SQL avec envois de mail journalier à l'administrateur système.

Tableau des VMs :

VM	RAM	vCPU	Disque	Rôle
redalert_ftp	1 Go	1	8 Go	Serveur FTP (ProFTPD)
redalert_web	1 Go	1	8 Go	Serveur Web (Nginx) avec auth basic
redalert_mariadb	2 Go	2	8 Go	Serveur Mariadb
redalert_control	1 Go	1	8 Go	Sans interface graphique. Python3, client MariaDB, client FTP, lib d'envoi de mail

Infrastructure :

- Hyperviseur : VMware Workstation
- Mode réseau : Bridged (les VM obtiennent une IP directement sur le réseau de la plateforme)
- Système : Debian 12 (bookworm), installation minimale sans interface graphique, uniquement "Utilitaires usuels du système" + "Serveur SSH"

Sécurisation SSH (identique sur les 3 VM) :

- Création du compte “monitor”, ajout au groupe “sudo”
- Authentification par clé SSH uniquement, mot de passe désactivé (“PasswordAuthentication no”)
- Connexion “root” interdite en SSH (“PermitRootLogin no”)
- Restriction explicite via “AllowUsers monitor” dans (/etc/ssh/sshd_config)
- “sudo” configuré sans mot de passe pour “monitor” (/etc/sudoers.d/monitor), (NOPASSWD:ALL)
  puissent exécuter des commandes en sudo sans terminal interactif

Extrait de configuration “/etc/ssh/sshd_config” modifiée :

```
monitor@redalertftp:~$ sudo sed -i \
> -e 's/^#\?PermitRootLogin.*/PermitRootLogin no/' \
>  -e 's/^#\?PasswordAuthentication.*/PasswordAuthentication no/' \
> -e 's/^#\?PubkeyAuthentication.*/PubkeyAuthentication yes/' \
> -e 's/^#\?ChallengeResponseAuthentication.*/ChallengeResponseAuthentication no/' \
>  /etc/ssh/sshd_config
monitor@redalertftp:~$ echo "AllowUsers monitor" | sudo tee -a /etc/ssh/sshd_config
AllowUsers monitor
monitor@redalertftp:~$ sudo sshd -t && sudo systemctl reload ssh
```

Service installé par VM :

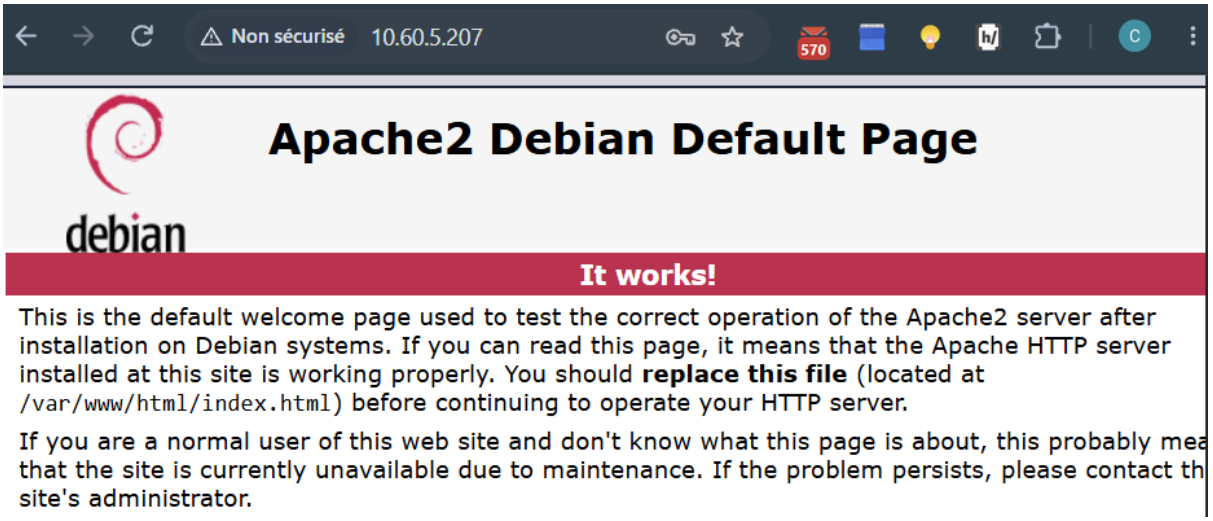
1. "redalert_ftp" (ProFTPD):

```
root@redalertftp:~# systemctl status proftpd
● proftpd.service - ProFTPD FTP Server
     Loaded: loaded (/lib/systemd/system/proftpd.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-09-18 10:40:10 CEST; 3 days ago
```

2. "redalert_web"(Apache + authentification basic):

```
monitor@redalertweb:~$ sudo a2enmod auth_basic authn_file authz_user
Considering dependency authn_core for auth_basic:
Module authn_core already enabled
Module auth_basic already enabled
Module authn_file already enabled
Considering dependency authz_core for authz_user:
Module authz_core already enabled
Module authz_user already enabled
monitor@redalertweb:~$ sudo htpasswd -c /etc/apache2/.htpasswd webuser
New password:
Re-type new password:
Adding password for user webuser
monitor@redalertweb:~$
```





3. "redalert_mariadb" (MariaDB):

```
monitor@redalertmariadb:~$ sudo systemctl enable --now mariadb
Synchronizing state of mariadb.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable mariadb
monitor@redalertmariadb:~$ sudo mysql_secure_installation
```

Objectif du Job 02 :

VM de contrôle (redalert_control)

J'ai installé une Debian minimale, sans environnement de bureau, puis les outils nécessaires au projet :

Outil	Paquet	Rôle
Python 3	python3	langage des scripts utilisés
Paramiko	python3-paramiko	connexion SSH depuis Python
Client MariaDB	mariadb-client	se connecter à une base
Client FTP	ftp	tester le serveur ftp en ligne de commande
Envoi de mails	smtplib	envoyer les rapports à l'administration

La VM a ensuite été placée sur le réseau de La Plateforme (IP 10.60.5.6)

Objectif du Job 03 : ***ssh_login.py***

se connecter en SSH à un des 3 serveurs depuis `redalert_control` et lancer une commande shell (`df`, `ls`...). Je l'écris avec `paramiko` (plus propre pour la suite que d'appeler le binaire `ssh` en sous-processus)

scripts/ssh_login.py utilise paramiko pour se connecter avec la clé monitor et exécuter une commande à distance. Il lit les IP des serveurs et le chemin de la clé depuis config.py.

Installation de python3-paramiko :

```
root@debian:~# sudo apt install python3-paramiko
```

Le script utilisé :

Il crée d'abord un client SSH. Puis il se connecte au serveur avec le compte "monitor" et une clé SSH (psmm_monitor). Ensuite il envoie la commande df -h qui affiche l'espace disque. Finalement il affiche le résultat puis ferme la connexion.

```python
import paramiko

ip = "10.60.5.206"
utilisateur = "monitor"
cle = "/home/monitor/.ssh/psmm_monitor"
passphrase = "root"

client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())

client.connect(ip, username=utilisateur, key_filename=cle, passphrase=passphrase)
print("Connecté au serveur !")

stdin, stdout, stderr = client.exec_command("df -h")

print(stdout.read().decode())
client.close()
```

```
monitor@debian:~$ python3 ~/psmm/ssh_login.py
Connecté au serveur !
Sys. de fichiers Taille Utilisé Dispo Uti% Monté sur
udev                948M       0  948M   0% /dev
tmpfs               194M    756K  193M   1% /run
/dev/sda1           6,9G    1,8G  4,8G  27% /
tmpfs               966M       0  966M   0% /dev/shm
tmpfs               5,0M       0  5,0M   0% /run/lock
tmpfs               194M       0  194M   0% /run/user/1000
tmpfs               194M       0  194M   0% /run/user/1001
```

Objectif du Job 04 :

prouver que le script peut lancer des commandes **avec les droits d'administrateur** (root) sur un serveur, à distance et sans que personne tape quoi que ce soit.

La commande est lancée avec **sudo -S**. L'option **-S** demande à sudo de lire le mot de passe sur l'entrée standard au lieu du clavier ;

Le script envoie le mot de passe de monitor avec **stdin.write().**

Nous avons testé avec sudo **-S whoami** et le serveur répond **root**, ce qui prouve que la commande s'exécute bien avec les droits administrateur.

```python
import paramiko

ip = "10.60.5.206"
utilisateur = "monitor"
cle = "/home/monitor/.ssh/psmm_monitor"
passphrase = "root"
mdp_sudo = "root"

client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())

client.connect(ip, username=utilisateur, key_filename=cle, passphrase=passphrase)
print("Connecté au serveur !")

stdin, stdout, stderr = client.exec_command("sudo -S whoami")
stdin.write(mdp_sudo + "\n")
stdin.flush()

print(stdout.read().decode())
client.close()
```

```
monitor@debian:~$ python3 ~/psmm/ssh_login_sudo.py
Connecté au serveur !
root
```

Objectif du Job 05 :

Le script se connecte en SSH au serveur redalert_mariadb (10.60.5.208) et lance : **sudo -S mysql -e "SHOW DATABASES;"**

Sur Debian, le compte root de MariaDB s'authentifie par le système (*unix_socket*). L'option -e exécute une requête SQL directement, sans ouvrir la console.

```python
import paramiko

ip = "10.60.5.208"
utilisateur = "monitor"
cle = "/home/monitor/.ssh/psmm_monitor"
passphrase = "root"
mdp_sudo = "root"

client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())

client.connect(ip, username=utilisateur, key_filename=cle, passphrase=passphrase)
print("Connecté au serveur !")

commande = 'sudo -S mysql -e "SHOW DATABASES;"'
stdin, stdout, stderr = client.exec_command(commande)
stdin.write(mdp_sudo + "\n")
stdin.flush()

print(stdout.read().decode())
print(stderr.read().decode())
client.close()
```

La liste des bases s'affiche, ce qui prouve que l'accès fonctionne.

```
monitor@debian:~$ python3 ~/psmm/ssh_mysql.py
Connecté au serveur !
Database
information_schema
mysql
performance_schema
sys
```

Objectif du Job 06 :

Récupérer dans les logs MariaDB les tentatives de connexion avec un mauvais compte ou mot de passe, puis les enregistrer dans une base avec le compte utilisé, la date/heure et l'IP.

Dans **/etc/mysql/mariadb.conf.d/50-server.cnf**, section **[mysqld]** :
On change **bind-adress** en **0.0.0.0** pour accepter les connexions venant du réseau, et pas seulement de localhost

```
bind-address            = 127.0.0.1
```

```
bind-address            = 0.0.0.0
```

On supprime **#** avant **log_error** pour l'activer et pour qu'il puisse écrire les erreurs dans un fichier



On ajoute **log_warnings = 2** pour enregistrer aussi les connexions refusées



Nous créons le dossier **/var/log/mysql** et le donnons à l'utilisateur **mysql**, pour que MariaDB puisse y écrire son fichier de log.



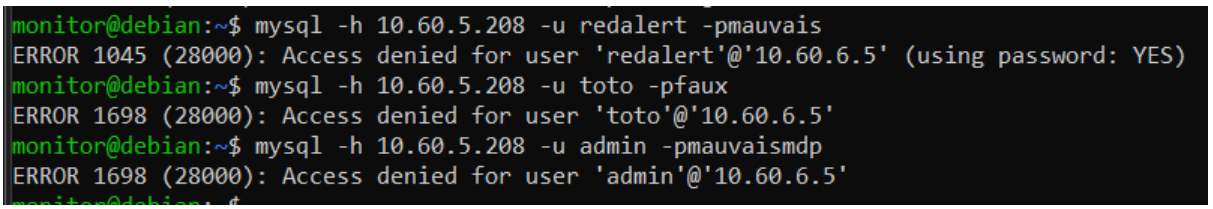
Après le redémarrage, nous regardons le log : MariaDB a bien démarré et il écrit dans le fichier.



Nous créons un **utilisateur "redalert"** accessible depuis le réseau. Sans lui, MariaDB bloquerait les connexions distantes sans les enregistrer dans le log.



Depuis la VM de contrôle, nous essayons de nous connecter avec de faux identifiants. MariaDB refuse chaque connexion.



Avec **grep**, nous affichons seulement les connexions refusées. Chaque ligne donne **la date/heure, le compte et l'IP**. C'est ce que le script utilise pour remplir la base.



Nous créons la base **psmm** et la table **mysql_errors**.
Elle contient 4 colonnes : **un numéro (id)**, **le compte utilisé**, **la date/heure et l'IP.** La ligne **UNIQUE** empêche d'enregistrer deux fois la même tentative.

```
MariaDB [(none)]> CREATE DATABASE psmm;
Query OK, 1 row affected (0,002 sec)

MariaDB [(none)]> USE psmm;
Database changed
MariaDB [psmm]> CREATE TABLE mysql_errors (
    -> id INT AUTO_INCREMENT PRIMARY KEY,
    -> utilisateur VARCHAR(100),
    -> date_heure DATETIME,
    -> ip VARCHAR(50),
    -> UNIQUE (utilisateur, date_heure, ip)
    -> );
Query OK, 0 rows affected (0,024 sec)
```

Le script se connecte en SSH au serveur MariaDB, puis lit le log avec **sudo grep "Access denied"** pour garder uniquement les connexions refusées. Pour chaque ligne, il récupère la date, le compte et l'IP, puis les enregistre dans la table **mysql_errors.**

```python
import paramiko

ip_serveur = "10.60.5.208"
utilisateur = "monitor"
cle = "/home/monitor/.ssh/psmm_monitor"
passphrase = "root"
mdp_sudo = "root"

def lancer(commande):
    stdin, stdout, stderr = client.exec_command("sudo -S " + commande)
    stdin.write(mdp_sudo + "\n")
    stdin.flush()
    return stdout.read().decode()

client = paramiko.SSHClient()
client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
client.connect(ip_serveur, username=utilisateur, key_filename=cle, passphrase=passphrase)
print("Connecté au serveur MariaDB !")

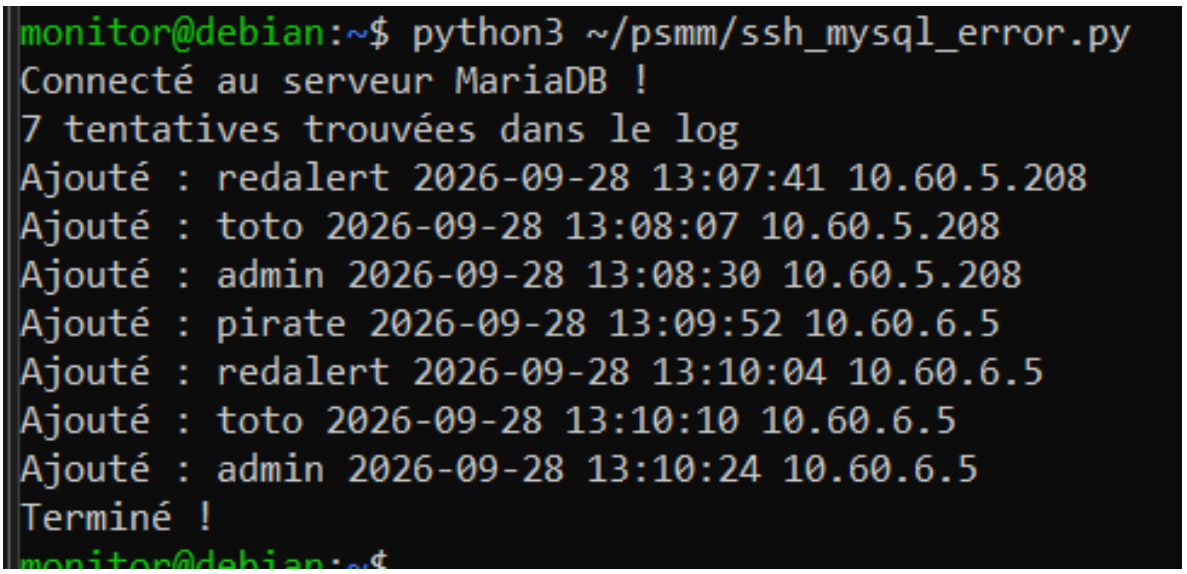
resultat = lancer('grep "Access denied" /var/log/mysql/error.log')
lignes = resultat.splitlines()
print(len(lignes), "tentatives trouvées dans le log")

for ligne in lignes:
    date_heure = ligne[0:19]
    morceaux = ligne.split("'")
    compte = morceaux[1]
    ip = morceaux[3]

    sql = f"INSERT IGNORE INTO mysql_errors (utilisateur, date_heure, ip) VALUES ('{compte}', '{date_heure}', '{ip}');"
    lancer('mysql psmm -e "' + sql + '"')
    print("Ajouté :", compte, date_heure, ip)

client.close()
print("Terminé !")
```

On exécute le script dans le terminal et comme on le voit sur la capture le script a bien marché



Objectif du job 07 :

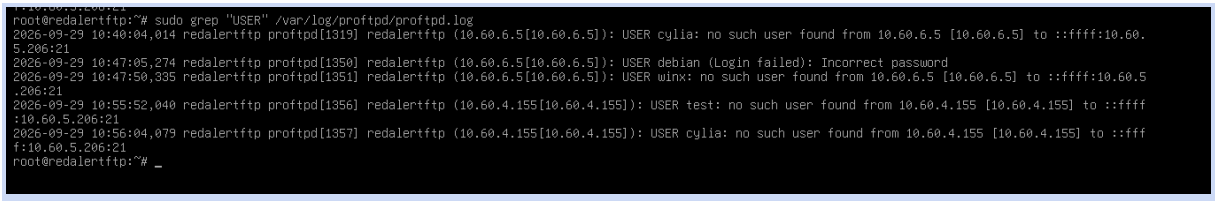
Récupérer les connexions ratées sur le serveur FTP et les enregistrer dans la base de données.

Le serveur FTP (ProFTPD) enregistre chaque tentative dans **/var/log/proftpd/proftpd.log.** Il y a deux types d'erreurs :

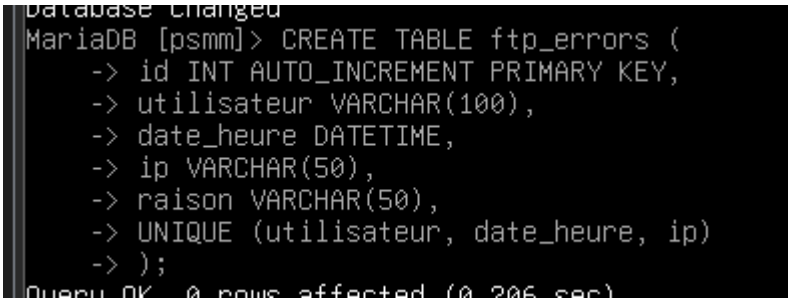
no such user found : le compte n'existe pas ;

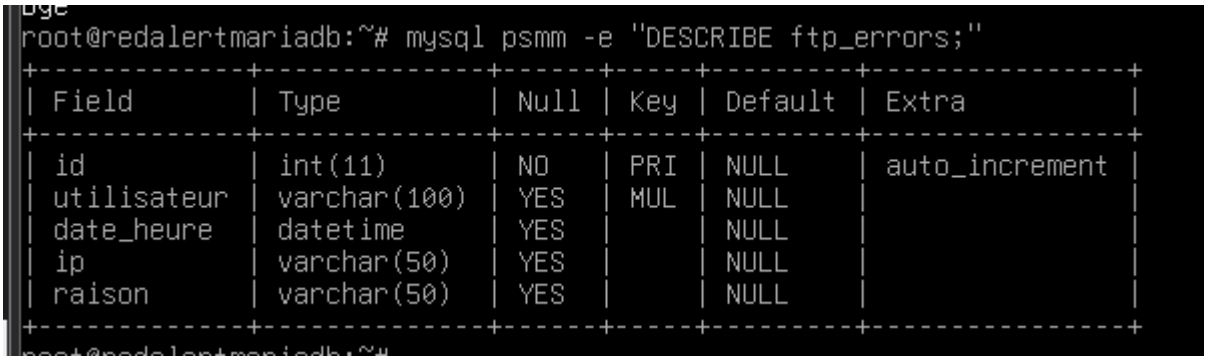
Incorrect password : le compte existe, mais le mot de passe est faux.

On voit aussi deux IP différentes : la VM de contrôle et notre PC.



Nous créons la table ftp_errors dans la base psmm. Elle stocke le compte, la date/heure, l'IP et la raison de l'échec.





Le script se connecte à deux serveurs : il lit le log sur le serveur FTP, puis il écrit dans la base sur le serveur MariaDB. Pour chaque erreur, il récupère la date, l'IP, le compte et la raison, puis les enregistre dans la table.

```python
import paramiko
ip_ftp = "172.20.10.5"
ip_mariadb = "172.20.10.6"
utilisateur = "monitor"
cle = "/home/monitor/.ssh/psmm_monitor"
passphrase = "root"
mdp_sudo = "root"

def connexion(ip):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    client.connect(ip, username=utilisateur, key_filename=cle, passphrase=passphrase)
    return client

def lancer(client, commande):
    stdin, stdout, stderr = client.exec_command("sudo -S " + commande)
    stdin.write(mdp_sudo + "\n")
    stdin.flush()
    return stdout.read().decode()

ftp = connexion(ip_ftp)
mariadb = connexion(ip_mariadb)
print("Connecté aux serveurs FTP et MariaDB !")

resultat = lancer(ftp, 'grep -e "no such user" -e "Incorrect password" /var/log/proftpd/proftpd.log')
lignes = resultat.splitlines()
print(len(lignes), "tentatives trouvées dans le log")

for ligne in lignes:
    date_heure = ligne[0:19]
    ip = ligne.split(" (")[1].split("[")[0]
    compte = ligne.split("USER ")[1].split(" ")[0].replace(":", "")

    if "no such user" in ligne:
        raison = "Utilisateur inconnu"
    else:
        raison = "Mot de passe incorrect"

    sql = f"INSERT IGNORE INTO ftp_errors (utilisateur, date_heure, ip, raison) VALUES ('{compte}', '{d
    lancer(mariadb, 'mysql psmm -e "' + sql + '"')
    print("Ajouté :", compte, date_heure, ip, raison)

ftp.close()
mariadb.close()
print("Terminé !")
```

exécution du script :

Le script affiche chaque tentative ajoutée dans la base.

```
monitor@debian:~$ python3 ~/psmm/ssh_ftp_error.py
Connecté aux serveurs FTP et MariaDB !
0 tentatives trouvées dans le log
Terminé !
```

SELECT * FROM ftp_errors :

La table contient toutes les tentatives ratées. Si on relance le script, aucun doublon n'est ajouté grâce à INSERT IGNORE.

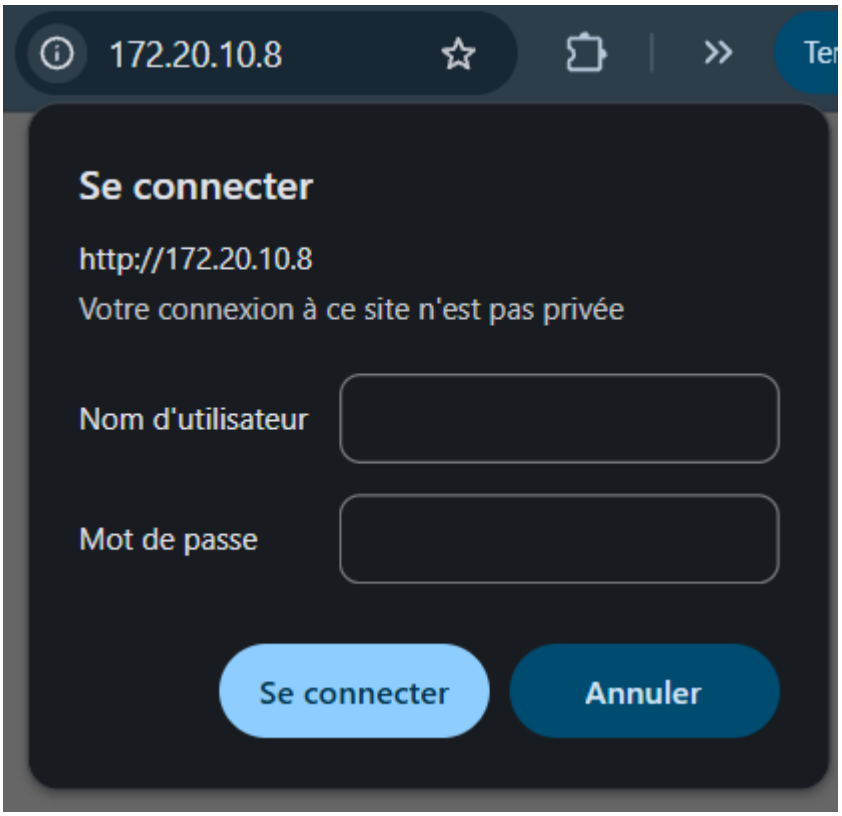
```
monitor@redalertmariadb:/home/debian$ sudo mysql psmm -e "SELECT * FROM ftp_errors;"
+----+-------------+---------------------+------------+-----------------------+
| id | utilisateur | date_heure          | ip         | raison                |
+----+-------------+---------------------+------------+-----------------------+
|  1 | dodida      | 2026-10-05 20:45:57 | 172.20.10.7 | Utilisateur inconnu   |
|  2 | lalala      | 2026-10-05 20:46:13 | 172.20.10.7 | Utilisateur inconnu   |
|  3 | cylia       | 2026-10-05 20:46:27 | 172.20.10.7 | Utilisateur inconnu   |
|  4 | debian      | 2026-10-05 20:46:40 | 172.20.10.7 | Mot de passe incorrect |
|  5 | test        | 2026-10-05 20:47:08 | 172.20.10.3 | Utilisateur inconnu   |
+----+-------------+---------------------+------------+-----------------------+
monitor@redalertmariadb:/home/debian$
```

Objectif du job 08 :

Récupérer les connexions ratées sur le site Web protégé par mot de passe et les enregistrer dans la base.

Fenêtre de connexion du site :

Le site Apache est protégé par une authentification basique : il faut un compte et un mot de passe pour y accéder.



Tentatives ratées (navigateur et curl) :

Depuis notre PC et depuis la VM de contrôle, nous essayons de nous connecter avec de faux identifiants. Le serveur refuse l'accès (401 Unauthorized).

```
monitor@debian:~$ curl -u admin:mauvais http://172.20.10.8
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>401 Unauthorized</title>
</head><body>
<h1>Unauthorized</h1>
<p>This server could not verify that you
are authorized to access the document
requested.  Either you supplied the wrong
credentials (e.g., bad password), or your
browser doesn't understand how to supply
the credentials required.</p>
<hr>
<address>Apache/2.4.68 (Debian) Server at 172.20.10.8 Port 80</address>
</body></html>
```

grep "auth_basic" dans le log Apache :

Apache enregistre chaque refus dans /var/log/apache2/error.log, avec la date, l'IP et le compte utilisé.

```
root@redalertweb:~# grep "auth_basic" /var/log/apache2/error.log
[Mon Oct 05 20:57:21.766968 2026] [auth_basic:error] [pid 903:tid 951] [client 172.20.10.3:64344] AH01618: user debian not found: /
[Mon Oct 05 21:56:50.094982 2026] [auth_basic:error] [pid 903:tid 945] [client 172.20.10.3:55070] AH01618: user debian not found: /
[Mon Oct 05 21:56:56.973651 2026] [auth_basic:error] [pid 903:tid 941] [client 172.20.10.3:57074] AH01618: user cylia not found: /
[Mon Oct 05 21:57:04.231717 2026] [auth_basic:error] [pid 903:tid 938] [client 172.20.10.3:59845] AH01618: user cylia not found: /
[Mon Oct 05 21:57:12.133995 2026] [auth_basic:error] [pid 903:tid 942] [client 172.20.10.3:62985] AH01618: user 123test not found: /
[Mon Oct 05 22:01:38.719130 2026] [auth_basic:error] [pid 903:tid 934] [client 172.20.10.7:40786] AH01618: user hacker not found: /
[Mon Oct 05 22:01:46.854772 2026] [auth_basic:error] [pid 903:tid 946] [client 172.20.10.7:54010] AH01618: user admin not found: /
[Mon Oct 05 22:02:13.769415 2026] [auth_basic:error] [pid 903:tid 950] [client 172.20.10.3:64084] AH01618: user debian not found: /
[Mon Oct 05 22:02:14.420062 2026] [auth_basic:error] [pid 903:tid 952] [client 172.20.10.3:64084] AH01618: user  not found: /
[Mon Oct 05 22:02:15.045252 2026] [auth_basic:error] [pid 903:tid 955] [client 172.20.10.3:64084] AH01618: user  not found: /
root@redalertweb:~#
```

DESCRIBE web_errors :

Nous créons la table web_errors, identique à ftp_errors.

```
monitor@redalertmariadb:/home/debian$ sudo mysql psmm -e "DESCRIBE web_errors;"
+-------------+--------------+------+-----+---------+----------------+
| Field       | Type         | Null | Key | Default | Extra          |
+-------------+--------------+------+-----+---------+----------------+
| id          | int(11)      | NO   | PRI | NULL    | auto_increment |
| utilisateur | varchar(100) | YES  | MUL | NULL    |                |
| date_heure  | datetime     | YES  |     | NULL    |                |
| ip          | varchar(50)  | YES  |     | NULL    |                |
| raison      | varchar(50)  | YES  |     | NULL    |                |
+-------------+--------------+------+-----+---------+----------------+
monitor@redalertmariadb:/home/debian$
```

code du script :

Le script lit le log sur le serveur Web, puis enregistre chaque tentative dans la base sur le serveur MariaDB. La date d'Apache est en anglais (Mon Oct 05 ...) : le script la convertit au format de MariaDB avec datetime.

```python
import paramiko
from datetime import datetime

cle = "/home/monitor/.ssh/psmm_monitor"
passphrase = "root"
mdp_sudo = "root"

web = paramiko.SSHClient()
web.set_missing_host_key_policy(paramiko.AutoAddPolicy())
web.connect("172.20.10.8", username="monitor", key_filename=cle, passphrase=passphrase)

bdd = paramiko.SSHClient()
bdd.set_missing_host_key_policy(paramiko.AutoAddPolicy())
bdd.connect("172.20.10.6", username="monitor", key_filename=cle, passphrase=passphrase)

print("Connecté aux deux serveurs !")

def lancer(serveur, commande):
    stdin, stdout, stderr = serveur.exec_command("sudo -S " + commande)
    stdin.write(mdp_sudo + "\n")
    stdin.flush()
    return stdout.read().decode()

resultat = lancer(web, 'grep "auth_basic:error" /var/log/apache2/error.log')
lignes = resultat.splitlines()
print(len(lignes), "tentatives trouvées dans le log")
```

```python
for ligne in lignes:
    date_texte = ligne[1:ligne.index("]")]
    date = datetime.strptime(date_texte, "%a %b %d %H:%M:%S.%f %Y")
    date_heure = date.strftime("%Y-%m-%d %H:%M:%S")

    ip = ligne.split("[client ")[1].split(":")[0]

    compte = ligne.split("user ")[1].split(":")[0].replace(" not found", "").strip()
    if compte == "":
        compte = "(vide)"

    if "not found" in ligne:
        raison = "Utilisateur inconnu"
    else:
        raison = "Mot de passe incorrect"

    sql = f"INSERT IGNORE INTO web_errors (utilisateur, date_heure, ip, raison) VALUES ('{compte}', '{d>
    lancer(bdd, 'mysql psmm -e "' + sql + '"')
    print("Ajouté :", compte, date_heure, ip, raison)

web.close()
bdd.close()
print("Terminé !")
```

exécution du script et SELECT * FROM web_errors :

Les 10 tentatives sont enregistrées. Un compte vide (formulaire validé sans nom) est noté (vide).

```
monitor@debian:~$ python3 ~/psmm/ssh_web_error.py
Connecté aux deux serveurs !
10 tentatives trouvées dans le log
Ajouté : debian 2026-10-05 20:57:21 172.20.10.3 Utilisateur inconnu
Ajouté : debian 2026-10-05 21:56:50 172.20.10.3 Utilisateur inconnu
Ajouté : cylia 2026-10-05 21:56:56 172.20.10.3 Utilisateur inconnu
Ajouté : cylia 2026-10-05 21:57:04 172.20.10.3 Utilisateur inconnu
Ajouté : 123test 2026-10-05 21:57:12 172.20.10.3 Utilisateur inconnu
Ajouté : hacker 2026-10-05 22:01:38 172.20.10.7 Utilisateur inconnu
Ajouté : admin 2026-10-05 22:01:46 172.20.10.7 Utilisateur inconnu
Ajouté : debian 2026-10-05 22:02:13 172.20.10.3 Utilisateur inconnu
Ajouté : (vide) 2026-10-05 22:02:14 172.20.10.3 Utilisateur inconnu
Ajouté : (vide) 2026-10-05 22:02:15 172.20.10.3 Utilisateur inconnu
Terminé !
```

Objectif du job 09 :

Envoyer chaque jour un mail à l'administrateur avec les tentatives de connexion ratées de la veille.

Mail de test :

Nous avons d'abord testé l'envoi d'un mail simple depuis la VM de contrôle avec la bibliothèque Python smtplib et un compte Gmail. Gmail impose un **mot de passe d'application** (avec la validation en deux étapes activée) pour qu'un script puisse envoyer des mails.

```python
import smtplib
from email.message import EmailMessage

expediteur = "cyliainfoamara22@gmail.com"
mdp_application = "0000000000000000"
destinataire = "ugo.rodriguez@laplateforme.io"

mail = EmailMessage()
mail["Subject"] = "Test PSMM"
mail["From"] = expediteur
mail["To"] = destinataire
mail.set_content("Bonjour, ceci est un test envoyé depuis la VM de contrôle.")

serveur = smtplib.SMTP_SSL("smtp.gmail.com", 465)
serveur.login(expediteur, mdp_application)
serveur.send_message(mail)
serveur.quit()

print("Mail envoyé !")
```

```
monitor@debian:~$ nano ~/psmm/test_mail.py
monitor@debian:~$ python3 ~/psmm/test_mail.py
Mail envoyé !
```

Code du script :

Le script se connecte au serveur MariaDB et récupère, dans les 3 tables (mysql_errors, ftp_errors, web_errors), les tentatives de la veille. Il les met en forme dans un texte, puis l'envoie par mail.

```python
import paramiko
import smtplib
from email.message import EmailMessage
from datetime import date, timedelta

cle = "/home/monitor/.ssh/psmm_monitor"
passphrase = "root"
mdp_sudo = "root"

expediteur = "cyliainfoamara22@gmail.com"
mdp_application = "000000000000000"
destinataire = "ugo.rodriguez@laplateforme.io"

jours = 1
jour = date.today() - timedelta(days=jours)

bdd = paramiko.SSHClient()
bdd.set_missing_host_key_policy(paramiko.AutoAddPolicy())
bdd.connect("172.20.10.6", username="monitor", key_filename=cle, passphrase=passphrase)


def lancer(serveur, commande):
    stdin, stdout, stderr = serveur.exec_command("sudo -S " + commande)
    stdin.write(mdp_sudo + "\n")
    stdin.flush()
    return stdout.read().decode()


texte = f"Rapport des tentatives de connexion échouées du {jour}\n"
tables = ["mysql_errors", "ftp_errors", "web_errors"]

for table in tables:
    sql = f"SELECT utilisateur, date_heure, ip FROM {table} WHERE DATE(date_heure) = '{jour}';"
    resultat = lancer(bdd, 'mysql -N psmm -e "' + sql + '"')
    lignes = resultat.splitlines()

    texte += f"\n=== {table} : {len(lignes)} tentative(s) ===\n"
    for ligne in lignes:
        texte += ligne.replace("\t", "  |  ") + "\n"
```

```
bdd.close()
print(texte)

mail = EmailMessage()
mail["Subject"] = f"[PSMM] Tentatives de connexion du {jour}"
mail["From"] = expediteur
mail["To"] = destinataire
mail.set_content(texte)

serveur = smtplib.SMTP_SSL("smtp.gmail.com", 465)
serveur.login(expediteur, mdp_application)
serveur.send_message(mail)
serveur.quit()

print("Mail envoyé !")
```

Exécution du script :

Le rapport affiche le nombre de tentatives par serveur, avec le compte, la date/heure et l'IP.

```
monitor@debian:~$ python3 ~/psmm/ssh_serveur_mail.py
Rapport des tentatives de connexion échouées du 2026-10-05

=== mysql_errors : 0 tentative(s) ===

=== ftp_errors : 5 tentative(s) ===
cylia  |  2026-10-05 20:46:27  |  172.20.10.7
debian  |  2026-10-05 20:46:40  |  172.20.10.7
dodida  |  2026-10-05 20:45:57  |  172.20.10.7
lalala  |  2026-10-05 20:46:13  |  172.20.10.7
test  |  2026-10-05 20:47:08  |  172.20.10.3

=== web_errors : 10 tentative(s) ===
(vide)  |  2026-10-05 22:02:14  |  172.20.10.3
(vide)  |  2026-10-05 22:02:15  |  172.20.10.3
123test  |  2026-10-05 21:57:12  |  172.20.10.3
admin  |  2026-10-05 22:01:46  |  172.20.10.7
cylia  |  2026-10-05 21:56:56  |  172.20.10.3
cylia  |  2026-10-05 21:57:04  |  172.20.10.3
debian  |  2026-10-05 20:57:21  |  172.20.10.3
debian  |  2026-10-05 21:56:50  |  172.20.10.3
debian  |  2026-10-05 22:02:13  |  172.20.10.3
hacker  |  2026-10-05 22:01:38  |  172.20.10.7

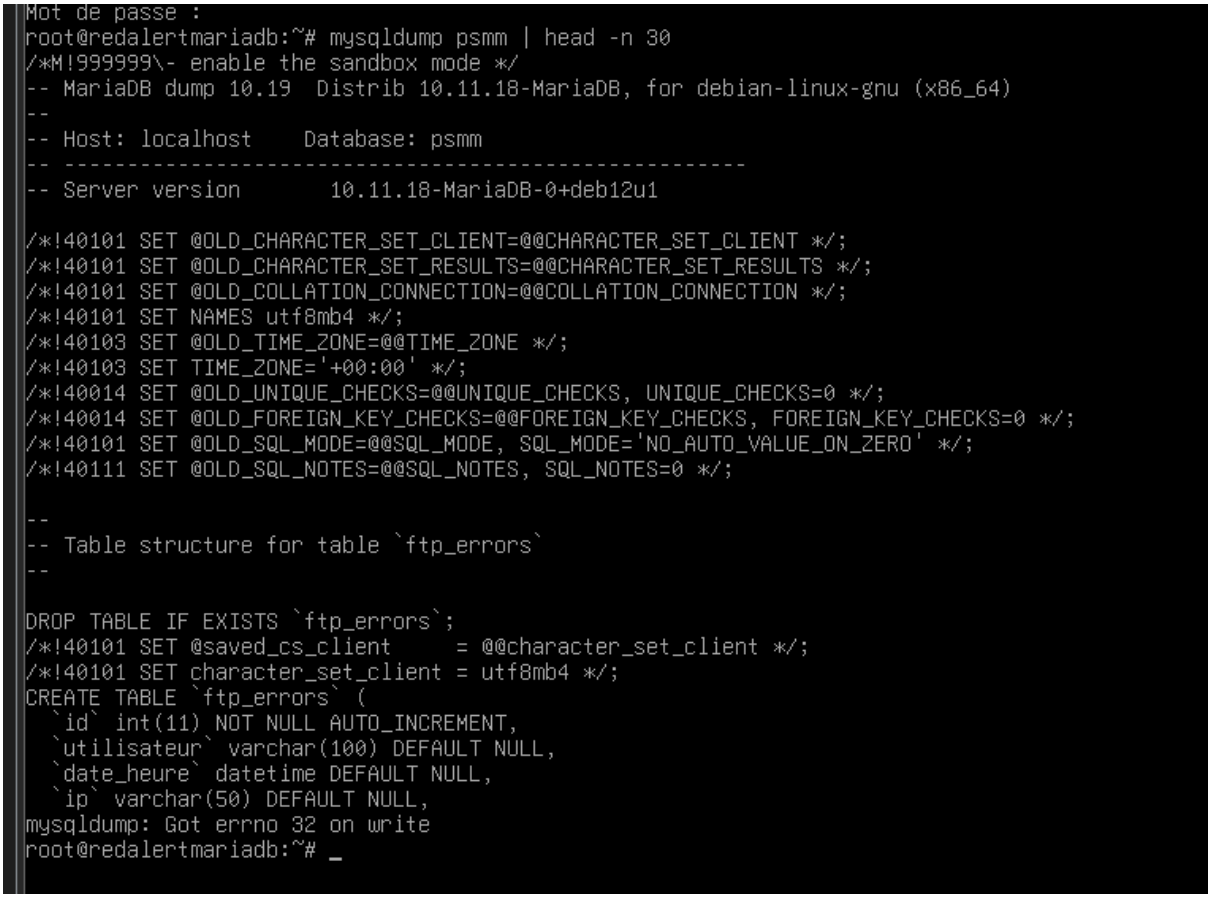
Mail envoyé !
```

Objectif du job 10 :

Faire une sauvegarde de la base psmm toutes les 3 heures. Chaque sauvegarde porte la date et l'heure dans son nom, et seules les 7 dernières sont conservées.

mysqldump psmm | head -n 30 :

Nous testons d'abord mysqldump sur le serveur MariaDB. Cet outil exporte la base dans un fichier texte .sql, qui contient toutes les commandes pour recréer les tables et leurs données. Le message errno 32 est normal : il vient de head, qui coupe l'affichage après 30 lignes.



Code du script :

Le script fonctionne en 3 étapes :

1. Il se connecte au serveur MariaDB et lance mysqldump.
2. Il enregistre le résultat sur la VM de contrôle, dans le dossier sauvegardes, avec un nom horodaté (par exemple psmm_2026-10-05_23-05-32.sql).
3. Il trie les sauvegardes par nom (donc par date) et supprime les plus anciennes pour n'en garder que 7.

```python
import paramiko
import os
from datetime import datetime

cle = "/home/monitor/.ssh/psmm_monitor"
passphrase = "root"
mdp_sudo = "root"
dossier = "/home/monitor/psmm/sauvegardes"
nb_a_garder = 7

os.makedirs(dossier, exist_ok=True)

bdd = paramiko.SSHClient()
bdd.set_missing_host_key_policy(paramiko.AutoAddPolicy())
bdd.connect("172.20.10.6", username="monitor", key_filename=cle, passphrase=passphrase)

stdin, stdout, stderr = bdd.exec_command("sudo -S mysqldump psmm")
stdin.write(mdp_sudo + "\n")
stdin.flush()
contenu = stdout.read().decode()
bdd.close()

horodatage = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
chemin = dossier + "/psmm_" + horodatage + ".sql"

with open(chemin, "w") as fichier:
    fichier.write(contenu)

print("Sauvegarde créée :", chemin)

sauvegardes = sorted(os.listdir(dossier))
while len(sauvegardes) > nb_a_garder:
    plus_ancienne = sauvegardes.pop(0)
    os.remove(dossier + "/" + plus_ancienne)
    print("Supprimée :", plus_ancienne)
```

```
monitor@debian:~$ python3 ~/psmm/ssh_cron_backup.py
Sauvegarde créée : /home/monitor/psmm/sauvegardes/psmm_2026-10-05_23-05-32.sql
```

Première sauvegarde (ls -l et head) :

Le script crée un premier fichier de sauvegarde. Avec head, nous vérifions qu'il contient bien la base.

```
monitor@debian:~$ ls -l ~/psmm/sauvegardes/
total 8
-rw-r--r-- 1 monitor monitor 5332  5 oct.  23:05 psmm_2026-10-05_23-05-32.sql
```

```
monitor@debian:~$ head ~/psmm/sauvegardes/psmm_*.sql
/*M!999999\- enable the sandbox mode */
-- MariaDB dump 10.19  Distrib 10.11.18-MariaDB, for debian-linux-gnu (x86_64)
--
-- Host: localhost    Database: psmm
-- ------------------------------------------------------
-- Server version       10.11.18-MariaDB-0+deb12u1

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
monitor@debian:~$
```

Test des 9 sauvegardes :

Nous lançons le script 9 fois de suite avec une boucle. À partir de la 8eme sauvegarde, le script supprime automatiquement les plus anciennes.

```
monitor@debian:~$ for i in 1 2 3 4 5 6 7 8 9; do python3 ~/psmm/ssh_cron_backup.py; sleep 1; done
Sauvegarde créée : /home/monitor/psmm/sauvegardes/psmm_2026-10-05_23-06-43.sql
Sauvegarde créée : /home/monitor/psmm/sauvegardes/psmm_2026-10-05_23-06-45.sql
Sauvegarde créée : /home/monitor/psmm/sauvegardes/psmm_2026-10-05_23-06-47.sql
Sauvegarde créée : /home/monitor/psmm/sauvegardes/psmm_2026-10-05_23-06-49.sql
Sauvegarde créée : /home/monitor/psmm/sauvegardes/psmm_2026-10-05_23-06-52.sql
Sauvegarde créée : /home/monitor/psmm/sauvegardes/psmm_2026-10-05_23-06-54.sql
Sauvegarde créée : /home/monitor/psmm/sauvegardes/psmm_2026-10-05_23-06-56.sql
Supprimée : psmm_2026-10-05_23-05-32.sql
Sauvegarde créée : /home/monitor/psmm/sauvegardes/psmm_2026-10-05_23-06-58.sql
Supprimée : psmm_2026-10-05_23-06-43.sql
Sauvegarde créée : /home/monitor/psmm/sauvegardes/psmm_2026-10-05_23-07-00.sql
Supprimée : psmm_2026-10-05_23-06-45.sql
monitor@debian:~$
```

ls du dossier sauvegardes :

Il reste bien 7 fichiers : les 7 sauvegardes les plus récentes.

```
monitor@debian:~$ ls ~/psmm/sauvegardes/
psmm_2026-10-05_23-06-47.sql  psmm_2026-10-05_23-06-54.sql  psmm_2026-10-05_23-07-00.sql
psmm_2026-10-05_23-06-49.sql  psmm_2026-10-05_23-06-56.sql
psmm_2026-10-05_23-06-52.sql  psmm_2026-10-05_23-06-58.sql
monitor@debian:~$
```

Crontab -l :

Nous planifions le script avec cron pour qu'il se lance automatiquement toutes les 3 heures.

```
# m h  dom mon dow   command
0 */3 * * * python3 /home/monitor/psmm/ssh_cron_backup.py
```

Objectif du job 11 :

Récupérer l'utilisation du CPU, de la RAM et du disque des 3 serveurs, la stocker dans la base et ne garder que les 72 dernières heures.

Les commandes testées à la main

Nous utilisons 3 commandes Linux :

- free -m donne la RAM totale et utilisée, avec laquelle on calcule le pourcentage ;
- df / donne le pourcentage du disque rempli ;
- vmstat donne le temps libre du processeur. L'utilisation du CPU vaut 100 − temps libre.

```
root@redalertweb:~# free -m | grep Mem
Mem:           1931         359        1547           1         168        1572
root@redalertweb:~# df / --output=pcent | tail -1
 27%
root@redalertweb:~# vmstat 1 2 | tail -1
 1  0      0 1584392  12500 160036    0    0     0     0  146   68  0  0 100  0  0
root@redalertweb:~# _
```

DESCRIBE system_status

La table system_status stocke, pour chaque mesure, le nom du serveur, la date/heure et les pourcentages CPU, RAM et disque.

```
root@redalertmariadb:~# sudo mysql psmm -e "DESCRIBE system_status;"
+------------+-------------+------+-----+---------+----------------+
| Field      | Type        | Null | Key | Default | Extra          |
+------------+-------------+------+-----+---------+----------------+
| id         | int(11)     | NO   | PRI | NULL    | auto_increment |
| serveur    | varchar(50) | YES  |     | NULL    |                |
| date_heure | datetime    | YES  |     | NULL    |                |
| cpu        | float       | YES  |     | NULL    |                |
| ram        | float       | YES  |     | NULL    |                |
| disque     | float       | YES  |     | NULL    |                |
+------------+-------------+------+-----+---------+----------------+
root@redalertmariadb:~#
```

Code du script

Le script se connecte à chaque serveur (FTP, MariaDB, Web), lance les 3 commandes, calcule les pourcentages et les enregistre dans la base. Il supprime ensuite les mesures de plus de 72 heures avec DELETE ... WHERE date_heure < NOW() - INTERVAL 72 HOUR.

```python
import paramiko
cle = "/home/monitor/.ssh/psmm_monitor"
passphrase = "root"
mdp_sudo = "root"

serveurs = {
    "ftp": "172.20.10.5",
    "mariadb": "172.20.10.6",
    "web": "172.20.10.8",
}

def connexion(ip):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    client.connect(ip, username="monitor", key_filename=cle, passphrase=passphrase)
    return client


def lancer(client, commande):
    stdin, stdout, stderr = client.exec_command(commande)
    return stdout.read().decode()


def lancer_sudo(client, commande):
    stdin, stdout, stderr = client.exec_command("sudo -S " + commande)
    stdin.write(mdp_sudo + "\n")
    stdin.flush()
    return stdout.read().decode()

bdd = connexion(serveurs["mariadb"])

for nom, ip in serveurs.items():
    client = connexion(ip)
    morceaux = lancer(client, "free -m | grep Mem").split()
    ram = round(int(morceaux[2]) / int(morceaux[1]) * 100, 1)
    disque = float(lancer(client, "df / --output=pcent | tail -1").strip().replace("%", ""))
    morceaux = lancer(client, "vmstat 1 2 | tail -1").split()
    cpu = 100 - int(morceaux[14])

    client.close()
```

```python
    client.close()

    sql = f"INSERT INTO system_status (serveur, date_heure, cpu, ram, disque) VALUES ('{nom}', NOW(), {
    lancer_sudo(bdd, 'mysql psmm -e "' + sql + '"')
    print(f"{nom} : CPU {cpu}% | RAM {ram}% | Disque {disque}%")

sql = "DELETE FROM system_status WHERE date_heure < NOW() - INTERVAL 72 HOUR;"
lancer_sudo(bdd, 'mysql psmm -e "' + sql + '"')

bdd.close()
print("Terminé !")
```

Exécution du script

Le script affiche l'état de chaque serveur.

```
monitor@debian:~$ nano ~/psmm/ssh_system_status.py
monitor@debian:~$ python3 ~/psmm/ssh_system_status.py
ftp : CPU 2% | RAM 17.0% | Disque 27.0%
mariadb : CPU 1% | RAM 24.0% | Disque 32.0%
web : CPU 1% | RAM 17.9% | Disque 27.0%
Terminé !
monitor@debian:~$
```

SELECT * FROM system_status

Les mesures sont enregistrées dans la base.

```
root@redalertmariadb:~# sudo mysql psmm -e "SELECT * FROM system_status;"
+----+---------+---------------------+------+------+--------+
| id | serveur | date_heure          | cpu  | ram  | disque |
+----+---------+---------------------+------+------+--------+
|  1 | ftp     | 2026-10-06 01:43:47 |    2 |   17 |     27 |
|  2 | mariadb | 2026-10-06 01:43:49 |    1 |   24 |     32 |
|  3 | web     | 2026-10-06 01:43:53 |    1 | 17.9 |     27 |
+----+---------+---------------------+------+------+--------+
root@redalertmariadb:~#
```

Crontab -l

Le script est lancé automatiquement toutes les 5 minutes, ce qui donne un historique de l'état des serveurs.

```
# m h  dom mon dow   command
*/5 * * * * python3 /home/monitor/psmm/ssh_system_status.py
```

Objectif du job 12 :

Envoyer un mail à l'administrateur si le CPU dépasse 70 %, la RAM 80 % ou le disque 90 %, avec une vérification toutes les 5 minutes.

Les seuils en haut du script

Les seuils sont regroupés en haut du script (SEUIL_CPU, SEUIL_RAM, SEUIL_DISQUE). On peut les modifier facilement, sans toucher au reste du code.

```
monitor@debian:~$ grep "^SEUIL" ~/psmm/ssh_system_mail.py
SEUIL_CPU = 70
SEUIL_RAM = 80
SEUIL_DISQUE = 90
```

Code du script

Le script reprend celui du Job 11. Après chaque mesure, il compare les valeurs aux seuils. Chaque dépassement est ajouté à une liste d'alertes. Si la liste n'est pas vide, un seul mail regroupe toutes les alertes

```python
import paramiko
import smtplib
from email.message import EmailMessage

SEUIL_CPU = 70
SEUIL_RAM = 80
SEUIL_DISQUE = 90


cle = "/home/monitor/.ssh/psmm_monitor"
passphrase = "root"
mdp_sudo = "root"
expediteur = "cyliainfoamara22@gmail.com"
mdp_application = "0000000000000000"
destinataire = "ugo.rodriguez@laplateforme.io"

serveurs = {
    "ftp": "172.20.10.5",
    "mariadb": "172.20.10.6",
    "web": "172.20.10.8",
}

def connexion(ip):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    client.connect(ip, username="monitor", key_filename=cle, passphrase=passphrase)
    return client


def lancer(client, commande):
    stdin, stdout, stderr = client.exec_command(commande)
    return stdout.read().decode()


def lancer_sudo(client, commande):
    stdin, stdout, stderr = client.exec_command("sudo -S " + commande)
    stdin.write(mdp_sudo + "\n")
    stdin.flush()
    return stdout.read().decode()

bdd = connexion(serveurs["mariadb"])
alertes = []
```

```
for nom, ip in serveurs.items():
    client = connexion(ip)
    morceaux = lancer(client, "free -m | grep Mem").split()
    ram = round(int(morceaux[2]) / int(morceaux[1]) * 100, 1)
    disque = float(lancer(client, "df / --output=pcent | tail -1").strip().replace("%", ""))
    morceaux = lancer(client, "vmstat 1 2 | tail -1").split()
    cpu = 100 - int(morceaux[14])

    client.close()

    sql = f"INSERT INTO system_status (serveur, date_heure, cpu, ram, disque) VALUES ('{nom}', NOW(), {cpu}, {ram}, {disque});"
    lancer_sudo(bdd, 'mysql psmm -e "' + sql + '"')
    print(f"{nom} : CPU {cpu}% | RAM {ram}% | Disque {disque}%")

    if cpu > SEUIL_CPU:
        alertes.append(f"{nom} : CPU à {cpu}% (seuil {SEUIL_CPU}%)")
    if ram > SEUIL_RAM:
        alertes.append(f"{nom} : RAM à {ram}% (seuil {SEUIL_RAM}%)")
    if disque > SEUIL_DISQUE:
        alertes.append(f"{nom} : Disque à {disque}% (seuil {SEUIL_DISQUE}%)")

sql = "DELETE FROM system_status WHERE date_heure < NOW() - INTERVAL 72 HOUR;"
lancer_sudo(bdd, 'mysql psmm -e "' + sql + '"')

bdd.close()

if len(alertes) > 0:
    texte = "Attention, des seuils ont été dépassés :\n\n"
    for alerte in alertes:
        texte += "- " + alerte + "\n"

    mail = EmailMessage()
    mail["Subject"] = "[PSMM] ALERTE ressources serveur"
    mail["From"] = expediteur
    mail["To"] = destinataire
    mail.set_content(texte)

    serveur_mail = smtplib.SMTP_SSL("smtp.gmail.com", 465)
    serveur_mail.login(expediteur, mdp_application)
    serveur_mail.send_message(mail)
    serveur_mail.quit()

    print(texte)
```

```
    print(texte)
    print("Mail d'alerte envoyé !")
else:
    print("Tout est normal, pas de mail.")
```

.

Mail d'alerte reçu

L'administrateur reçoit le mail avec la liste des serveurs concernés et les valeurs mesurées.

Test avec les vrais seuils

Avec les vrais seuils, les serveurs sont au repos : le script affiche « Tout est normal » et n'envoie pas de mail.

```
monitor@debian:~$ nano ~/psmm/ssh_system_mail.py
monitor@debian:~$ python3 ~/psmm/ssh_system_mail.py
ftp : CPU 0% | RAM 17.2% | Disque 27.0%
mariadb : CPU 2% | RAM 23.4% | Disque 32.0%
web : CPU 1% | RAM 18.0% | Disque 27.0%
Tout est normal, pas de mail.
```

Crontab -l

Le script est planifié toutes les 5 minutes. Il remplace celui du Job 11, puisqu'il enregistre lui aussi les mesures dans la base.

```
# m h  dom mon dow   command
*/5 * * * * python3 /home/monitor/psmm/ssh_system_mail.py
```

Objectif du job 13 :

Le script tourne toutes les 5 minutes. Si un serveur reste surchargé, l'administrateur recevrait 12 mails par heure. Il faut limiter l'envoi à un mail par heure maximum.

Code du script

Le script enregistre l'heure de chaque mail envoyé dans un fichier (dernier_mail.txt). Avant d'envoyer une alerte, il lit ce fichier : si le dernier mail date de moins d'une heure (3 600 secondes), il n'envoie rien.

```
GNU nano 7.2                    /home/monitor/psmm/ssh_system_mail_in.py
import paramiko
import smtplib
import os
import time
from email.message import EmailMessage

SEUIL_CPU = 70
SEUIL_RAM = 80
SEUIL_DISQUE = 90

FICHIER_DERNIER_MAIL = "/home/monitor/psmm/dernier_mail.txt"
DELAI = 3600

cle = "/home/monitor/.ssh/psmm_monitor"
passphrase = "root"
mdp_sudo = "root"
expediteur = "cyliainfoamara22@gmail.com"
mdp_application = "0000000000000"
destinataire = "ugo.rodriguez@laplateforme.io"

serveurs = {
    "ftp": "172.20.10.5",
    "mariadb": "172.20.10.6",
    "web": "172.20.10.8",
}

def connexion(ip):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    client.connect(ip, username="monitor", key_filename=cle, passphrase=passphrase)
    return client


def lancer(client, commande):
    stdin, stdout, stderr = client.exec_command(commande)
    return stdout.read().decode()


def lancer_sudo(client, commande):
    stdin, stdout, stderr = client.exec_command("sudo -S " + commande)
    stdin.write(mdp_sudo + "\n")
    stdin.flush()
    return stdout.read().decode()

bdd = connexion(serveurs["mariadb"])
```

```
bdd = connexion(serveurs["mariadb"])
alertes = []

for nom, ip in serveurs.items():
    client = connexion(ip)
    morceaux = lancer(client, "free -m | grep Mem").split()
    ram = round(int(morceaux[2]) / int(morceaux[1]) * 100, 1)
    disque = float(lancer(client, "df / --output=pcent | tail -1").strip().replace("%", ""))
    morceaux = lancer(client, "vmstat 1 2 | tail -1").split()
    cpu = 100 - int(morceaux[14])

    client.close()

    sql = f"INSERT INTO system_status (serveur, date_heure, cpu, ram, disque) VALUES ('{nom}', NO
    lancer_sudo(bdd, 'mysql psmm -e "' + sql + '"')
    print(f"{nom} : CPU {cpu}% | RAM {ram}% | Disque {disque}%")

    if cpu > SEUIL_CPU:
        alertes.append(f"{nom} : CPU à {cpu}% (seuil {SEUIL_CPU}%)")
    if ram > SEUIL_RAM:
        alertes.append(f"{nom} : RAM à {ram}% (seuil {SEUIL_RAM}%)")
    if disque > SEUIL_DISQUE:
        alertes.append(f"{nom} : Disque à {disque}% (seuil {SEUIL_DISQUE}%)")

sql = "DELETE FROM system_status WHERE date_heure < NOW() - INTERVAL 72 HOUR;"
lancer_sudo(bdd, 'mysql psmm -e "' + sql + '"')

bdd.close()

if len(alertes) > 0:
    maintenant = time.time()
    dernier_mail = 0
    if os.path.exists(FICHIER_DERNIER_MAIL):
        with open(FICHIER_DERNIER_MAIL, "r") as fichier:
            dernier_mail = float(fichier.read())

    if maintenant - dernier_mail >= DELAI:
        texte = "Attention, des seuils ont été dépassés :\n\n"
        for alerte in alertes:
            texte += "- " + alerte + "\n"

        mail = EmailMessage()
        mail["Subject"] = "[PSMM] ALERTE ressources serveur"
        mail["From"] = expediteur
```

```
        mail = EmailMessage()
        mail["Subject"] = "[PSMM] ALERTE ressources serveur"
        mail["From"] = expediteur
        mail["To"] = destinataire
        mail.set_content(texte)

        serveur_mail = smtplib.SMTP_SSL("smtp.gmail.com", 465)
        serveur_mail.login(expediteur, mdp_application)
        serveur_mail.send_message(mail)
        serveur_mail.quit()

        with open(FICHIER_DERNIER_MAIL, "w") as fichier:
            fichier.write(str(maintenant))

        print(texte)
        print("Mail d'alerte envoyé !")
    else:
        attente = int((DELAI - (maintenant - dernier_mail)) / 60)
        print(f"Alerte détectée, mais un mail a déjà été envoyé il y a moins d'1 heure.")
        print(f"Prochain mail possible dans {attente} minutes.")
else:
    print("Tout est normal, pas de mail.")
```

Premier lancement

Pour tester, nous avons baissé le seuil RAM à 10 %. Les 3 serveurs le dépassent, et le mail d'alerte est envoyé.

```
monitor@debian:~$ python3 ~/psmm/ssh_system_mail_1h.py
ftp : CPU 1% | RAM 17.6% | Disque 27.0%
mariadb : CPU 0% | RAM 23.9% | Disque 32.0%
web : CPU 1% | RAM 17.8% | Disque 27.0%
Attention, des seuils ont été dépassés :

- ftp : RAM à 17.6% (seuil 10%)
- mariadb : RAM à 23.9% (seuil 10%)
- web : RAM à 17.8% (seuil 10%)

Mail d'alerte envoyé !
```

Deuxième lancement

Relancé tout de suite, le script détecte les mêmes alertes, mais n'envoie pas de mail. Il indique le temps restant avant le prochain envoi possible.

```
monitor@debian:~$ python3 ~/psmm/ssh_system_mail_1h.py
ftp : CPU 0% | RAM 17.6% | Disque 27.0%
mariadb : CPU 0% | RAM 23.9% | Disque 32.0%
web : CPU 0% | RAM 17.9% | Disque 27.0%
Alerte détectée, mais un mail a déjà été envoyé il y a moins d'1 heure.
Prochain mail possible dans 58 minutes.
```

Crontab -l

Le script est planifié toutes les 5 minutes, à la place de celui du Job 12.

```
# m h  dom mon dow   command
*/5 * * * * python3 /home/monitor/psmm/ssh_system_mail_1h.py
```

Objectif du job 14 :

Sudo apt update

apt update télécharge la liste des dernières versions des logiciels. Il indique combien de paquets peuvent être mis à jour.

```
root@redalertweb:~# sudo apt update
Atteint :1 http://deb.debian.org/debian bookworm InRelease
Atteint :2 http://security.debian.org/debian-security bookworm-security InRelease
Atteint :3 http://deb.debian.org/debian bookworm-updates InRelease
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Tous les paquets sont à jour.
```

Apt list --upgradable

Cette commande affiche la liste des logiciels qui ont une nouvelle version disponible.

```
root@redalertweb:~# apt list --upgradable
En train de lister... Fait
```

Sudo apt upgrade -y

apt upgrade installe les mises à jour. L'option -y répond « oui » automatiquement, ce qui est nécessaire dans un script.

```
root@redalertweb:~# sudo apt upgrade -y
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Calcul de la mise à jour... Fait
0 mis à jour, 0 nouvellement installés, 0 à enlever et 0 non mis à jour.
root@redalertweb:~# _
```

Ls /var/run/reboot-required

Après certaines mises à jour (par exemple le noyau Linux), Debian crée le fichier /var/run/reboot-required pour signaler qu'un redémarrage est nécessaire. S'il n'existe pas, aucun redémarrage n'est nécessaire.

```
0 mis à jour, 0 nouvellement installés, 0 à enlever et 0 non mis à jour.
root@redalertweb:~# ls /var/run/reboot-required
ls: impossible d'accéder à '/var/run/reboot-required': Aucun fichier ou dossier de ce type
root@redalertweb:~#
```

Code du script

Pour chaque serveur, le script :

1. se connecte à Alcasar ;
2. rafraîchit la liste des mises à jour (apt-get update) ;
3. compte les mises à jour disponibles avec une simulation (apt-get -s upgrade) ;
4. les installe s'il y en a ;
5. vérifie si le fichier reboot-required existe ;
6. se déconnecte d'Alcasar.

Si au moins un serveur doit redémarrer, il envoie un mail à l'administrateur.

```python
import paramiko
import smtplib
from email.message import EmailMessage

cle = "/home/monitor/.ssh/psmm_monitor"
passphrase = "root"
mdp_sudo = "root"
expediteur = "cyliainfoamara22@gmail.com"
mdp_application = "0000000000000000"
destinataire = "ugo.rodriguez@laplateforme.io"

serveurs = {
    "ftp": "172.20.10.5",
    "mariadb": "172.20.10.6",
    "web": "172.20.10.8",
}


def connexion(ip):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    client.connect(ip, username="monitor", key_filename=cle, passphrase=passphrase)
    return client


def lancer(client, commande):
    stdin, stdout, stderr = client.exec_command(commande)
    return stdout.read().decode()


def lancer_sudo(client, commande):
    stdin, stdout, stderr = client.exec_command("sudo -S " + commande)
    stdin.write(mdp_sudo + "\n")
    stdin.flush()
    return stdout.read().decode()


def connexion_alcasar(client):
    print("   (connexion Alcasar : à faire)")


def deconnexion_alcasar(client):
    print("   (déconnexion Alcasar : à faire)")
```

```
a_redemarrer = []

for nom, ip in serveurs.items():
    print(f"=== {nom} ({ip}) ===")
    client = connexion(ip)


    connexion_alcasar(client)
    lancer_sudo(client, "apt-get update")
    nb = int(lancer(client, "apt-get -s upgrade | grep -c '^Inst'").strip())
    print(f"   {nb} mise(s) à jour disponible(s)")

    if nb > 0:
        lancer_sudo(client, "DEBIAN_FRONTEND=noninteractive apt-get upgrade -y")
        print("   Mises à jour installées")

    reponse = lancer(client, "test -f /var/run/reboot-required && echo OUI || echo NON").strip()
    if reponse == "OUI":
        print("   Redémarrage nécessaire !")
        a_redemarrer.append(nom)
    else:
        print("   Pas de redémarrage nécessaire")

    deconnexion_alcasar(client)
    client.close()


if len(a_redemarrer) > 0:
    texte = "Les mises à jour ont été installées.\n"
    texte += "Les serveurs suivants doivent être redémarrés :\n\n"
    for nom in a_redemarrer:
        texte += "- " + nom + "\n"

    mail = EmailMessage()
    mail["Subject"] = "[PSMM] Redémarrage nécessaire après mise à jour"
    mail["From"] = expediteur
    mail["To"] = destinataire
    mail.set_content(texte)

    serveur_mail = smtplib.SMTP_SSL("smtp.gmail.com", 465)
    serveur_mail.login(expediteur, mdp_application)
    serveur_mail.send_message(mail)
    serveur_mail.quit()
    print("\nMail envoyé à l'administrateur !")
```

```
    print("\nMail envoyé à l'administrateur !")
else:
    print("\nAucun serveur à redémarrer, pas de mail.")
```

Exécution du script

Le script affiche, pour chaque serveur, le nombre de mises à jour et si un redémarrage est nécessaire.

```
monitor@debian:~$ python3 ~/psmm/ssh_update.py
=== ftp (172.20.10.5) ===
   (connexion Alcasar : à faire)
   11 mise(s) à jour disponible(s)
   Mises à jour installées
   Pas de redémarrage nécessaire
   (déconnexion Alcasar : à faire)
=== mariadb (172.20.10.6) ===
   (connexion Alcasar : à faire)
   5 mise(s) à jour disponible(s)
   Mises à jour installées
   Pas de redémarrage nécessaire
   (déconnexion Alcasar : à faire)
=== web (172.20.10.8) ===
   (connexion Alcasar : à faire)
   0 mise(s) à jour disponible(s)
   Pas de redémarrage nécessaire
   (déconnexion Alcasar : à faire)

Aucun serveur à redémarrer, pas de mail.
```

Test du mail (touch /var/run/reboot-required)

Pour tester l'envoi du mail, nous avons créé le fichier reboot-required à la main sur un serveur. Le script détecte qu'un redémarrage est nécessaire et prévient l'administrateur.

Objectif du job 15 :

Envoyer périodiquement dans un espace Google Chat un message qui détaille l'état des serveurs.

L'espace Google Chat

Nous avons créé un espace Google Chat « redalert - PSMM » avec les membres du groupe et notre accompagnateur pédagogique.

Création du webhook

Dans l'espace, nous avons ajouté un webhook. Il fournit une adresse secrète : tout message envoyé à cette adresse apparaît dans le chat.

Test simple

Nous avons testé l'envoi d'un premier message depuis la VM de contrôle avec la bibliothèque Python urllib. Le message est envoyé au format JSON.

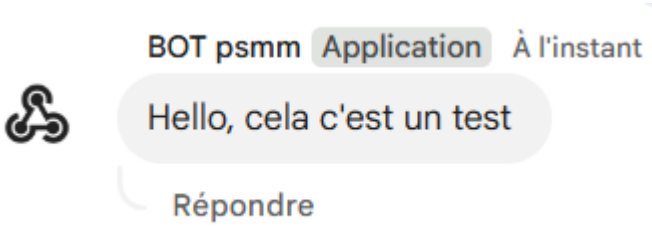
```
 GNU nano 7.2                                  /home/monitor/psmm/test_chat.p
import json
import urllib.request

webhook = "blablablacar"

message = {"text": "Hello, cela c'est un test "}

requete = urllib.request.Request(
    webhook,
    data=json.dumps(message).encode(),
    headers={"Content-Type": "application/json; charset=UTF-8"},
)
urllib.request.urlopen(requete)

print("Message envoyé dans le chat !")
```



Code du script

Le script se connecte à la base de données et récupère :

- la dernière mesure CPU, RAM et disque de chaque serveur (Jobs 11-12) ;
- le nombre de tentatives de connexion ratées du jour sur chaque serveur (Jobs 06, 07, 08).

Il met ces informations en forme et les envoie dans le chat.

```python
import paramiko
import json
import urllib.request
from datetime import datetime

cle = "/home/monitor/.ssh/psmm_monitor"
passphrase = "root"
mdp_sudo = "root"
webhook = "tiktokshop"


def lancer_sudo(client, commande):
    stdin, stdout, stderr = client.exec_command("sudo -S " + commande)
    stdin.write(mdp_sudo + "\n")
    stdin.flush()
    return stdout.read().decode()


bdd = paramiko.SSHClient()
bdd.set_missing_host_key_policy(paramiko.AutoAddPolicy())
bdd.connect("172.20.10.6", username="monitor", key_filename=cle, passphrase=passphrase)

maintenant = datetime.now().strftime("%d/%m/%Y à %H:%M")
texte = f"*📊 Rapport des serveurs redalert — {maintenant}*\n\n"

texte += "*État des serveurs :*\n"
sql = "SELECT serveur, cpu, ram, disque FROM system_status ORDER BY id DESC LIMIT 3;"
resultat = lancer_sudo(bdd, 'mysql -N psmm -e "' + sql + '"')

for ligne in resultat.splitlines():
    morceaux = ligne.split("\t")
    nom, cpu, ram, disque = morceaux[0], morceaux[1], morceaux[2], morceaux[3]
    texte += f"• {nom} : CPU {cpu}% | RAM {ram}% | Disque {disque}%\n"

texte += "\n*Tentatives de connexion ratées aujourd'hui :*\n"
tables = {"MariaDB": "mysql_errors", "FTP": "ftp_errors", "Web": "web_errors"}

for nom, table in tables.items():
    sql = f"SELECT COUNT(*) FROM {table} WHERE DATE(date_heure) = CURDATE();"
    nombre = lancer_sudo(bdd, 'mysql -N psmm -e "' + sql + '"').strip()
    texte += f"• {nom} : {nombre}\n"

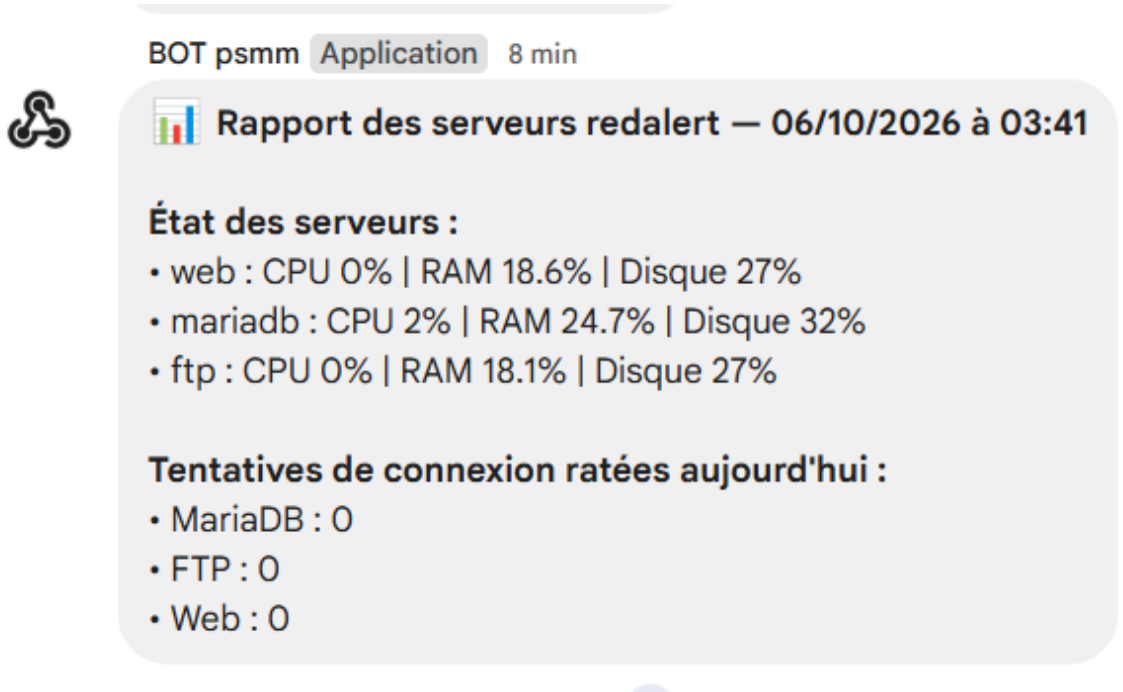
bdd.close()
print(texte)
```

```python
message = {"text": texte}
requete = urllib.request.Request(
    webhook,
    data=json.dumps(message).encode(),
    headers={"Content-Type": "application/json; charset=UTF-8"},
)
urllib.request.urlopen(requete)

print("Message envoyé dans le chat !")
```

Message reçu dans le chat

Le rapport apparaît dans l'espace, lisible par tout le groupe.



Crontab -l

Le script est planifié toutes les heures (0 * * * *).

```
# m h  dom mon dow   command
0 * * * * python3 /home/monitor/psmm/ssh_chat.py
```